12. 병렬 처리 및 메모리 최적화 분석

개요

Stwo 프로젝트는 Circle STARK 증명 시스템의 성능을 극대화하기 위해 Rayon을 활용한 병렬 처리와 다양한 메모리 최적화 기법을 구현합니다. 이 문서는 Stwo에서 사용되는 병렬 처리 전략, 메모리 접근 패턴 최적화, 캐시 효율성 향상 기법, 그리고 대규모 증명 생성을 위한 확장성 설계를 상세히 분석합니다.

Rayon 기반 병렬 처리 구현

병렬 처리 매크로

Stwo는 조건부 병렬 처리를 위한 매크로를 제공하여 `parallel` 피처 플래그에 따라 순차 또는 병렬 실행을 선택합니다:

```
#[macro_export]
macro_rules! parallel_iter {
($i: expr) => {{
#[cfg(not(feature = "parallel"))]
let iter = $i.into_iter();

#[cfg(feature = "parallel")]
let iter = $i.into_par_iter();
```

```
iter
}};
}
```

이 매크로는 컴파일 타임에 병렬 처리 여부를 결정하여 성능과 호환성을 모두 보장합니다.

피처 기반 병렬 처리 활성화

Cargo.toml에서 병렬 처리 기능을 선택적으로 활성화할 수 있습니다:

```
[features]
parallel = ["rayon", "indexmap/rayon"]

[dependencies]
rayon = { workspace = true, optional = true }
```

병렬 처리 적용 영역

FFT 벡터 전치 병렬화

FFT에서 사용되는 벡터 전치 연산은 병렬 처리로 최적화됩니다:

```
pub unsafe fn transpose_vecs(values: *mut u32, log_n_vecs: usize) {
let half = log_n_vecs / 2;
let values = UnsafeMut(values);
parallel_iter!(0..1 << half).for_each(|a| {
let values = values.get();
```

```
for b in 0..1 << (log_n_vecs & 1) {
for c in 0..1 << half {
let i = (a << (log_n_vecs - half)) | (b << half) | c;
let j = (c << (log_n_vecs - half)) | (b << half) | a;
if i >= j { continue; }
// 병렬로 벡터 교환 수행
let val0 = load(values.add(i << 4).cast_const());
let val1 = load(values.add(j << 4).cast_const());
store(values.add(i << 4), val1);
store(values.add(j << 4), val0);
}
}
});
}
```

비트 역순 병렬화

대용량 데이터의 비트 역순 연산은 병렬 처리로 가속화됩니다:

```
pub fn bit_reverse_m31(data: &mut [PackedBaseField]) {
let log_size = data.len().ilog2();
let a_bits = log_size - 2 * W_BITS - VEC_BITS;
let data = UnsafeMut(data);
parallel_iter!(0u32..(1 << a_bits)).for_each(|a| {
let data = unsafe { data.get() };
```

```
for w_l in 0u32..1 << W_BITS {
let w_l_rev = w_l.reverse_bits() >> (u32::BITS - W_BITS);
for w_h in 0..w_l_rev + 1 {
// 병렬로 16x16 블록 처리
let idx = ((((w_h << a_bits) | a) << W_BITS) | w_l) as usize;
let idx_rev = bit_reverse_index(idx, log_size - VEC_BITS);
if idx > idx_rev { continue; }
// 블록 단위 병렬 처리
let chunk0 = array::from_fn(|i| unsafe {
*data.get_unchecked_mut(idx + (i << (2 * W_BITS + a_bits)))
});
let values0 = bit_reverse16(chunk0);
// 결과 저장
for i in 0..16 {
unsafe {
*data.get_unchecked_mut(idx + (i << (2 * W_BITS + a_bits))) =
values0[i];
}
}
}
}
});
```

```
}
```

배치 역원 계산 병렬화

대용량 필드 원소들의 배치 역원 계산은 청크 단위로 병렬 처리됩니다:

```
pub fn batch_inverse_chunked<T: FieldExpOps + Send + Sync>(
column: &[T],
dst: &mut [T],
chunk_size: usize,
) {
assert!(column.len() <= dst.len());

#[cfg(not(feature = "parallel"))]
let iter = dst.chunks_mut(chunk_size).zip(column.chunks(chunk_size));

#[cfg(feature = "parallel")]
let iter = dst
.par_chunks_mut(chunk_size)
.zip(column.par_chunks(chunk_size));

iter.for_each(|(dst, column)| {
batch_inverse_in_place(column, dst);
});
```

```
}
```

메모리 최적화 기법

안전하지 않은 메모리 접근 최적화

성능 향상을 위해 안전하지 않은 메모리 접근을 활용하되, 병렬 처리에서 안전성을 보장합니다:

```
pub struct UnsafeMut<T: ?Sized>(pub *mut T);
impl<T: ?Sized> UnsafeMut<T> {
/// # Safety
/// 원시 가변 포인터를 반환합니다.
pub const unsafe fn get(&self) -> *mut T {
self.0
}
}

unsafe impl<T: ?Sized> Send for UnsafeMut<T> {}
unsafe impl<T: ?Sized> Sync for UnsafeMut<T> {}

pub struct UnsafeConst<T>(pub *const T);
impl<T> UnsafeConst<T> {
/// # Safety
/// 원시 상수 포인터를 반환합니다.
```

```
pub const unsafe fn get(&self) -> *const T {
self.0
}
}

unsafe impl<T> Send for UnsafeConst<T> {}
unsafe impl<T> Sync for UnsafeConst<T> {}
```

초기화되지 않은 벡터 최적화

메모리 할당 성능을 향상시키기 위해 초기화되지 않은 벡터를 사용합니다:

```
/// # Safety
/// 호출자는 사용 전에 벡터가 초기화되었는지 확인해야 합니다.
#[allow(clippy::uninit_vec)]
pub unsafe fn uninit_vec<T>(len: usize) -> Vec<T> {
let mut vec = Vec::with_capacity(len);
vec.set_len(len);
vec
}
```

이 함수는 다음과 같은 상황에서 사용됩니다:

```
impl Column<BaseField> for BaseColumn {
#[allow(clippy::uninit_vec)]
```

```
unsafe fn uninitialized(length: usize) -> Self {
let mut data = Vec::with_capacity(length.div_ceil(N_LANES));
data.set_len(length.div_ceil(N_LANES));
Self { data, length }
}
}
```

메모리 레이아웃 최적화

패킹된 필드 구조

메모리 효율성을 위해 필드 원소들을 패킹하여 저장합니다:

```
#[derive(Clone, Debug)]
pub struct BaseColumn {
pub data: Vec<PackedBaseField>,
/// BaseField 값들의 개수
pub length: usize,
}

impl BaseColumn {
/// 전체 BaseField 벡터를 포함하는 슬라이스를 추출합니다.
pub fn as_slice(&self) -> &[BaseField] {
&cast_slice(&self.data)[..self.length]
}
```

```
/// 전체 BaseField 벡터를 포함하는 가변 슬라이스를 추출합니다.
pub fn as_mut_slice(&mut self) -> &mut [BaseField] {
&mut cast_slice_mut(&mut self.data)[..self.length]
}
}
```

좌표별 보안 필드 저장

보안 필드는 좌표별로 분리하여 저장하여 메모리 접근 패턴을 최적화합니다:

```
impl SecureColumn {
// 단일 PackedSecureField 원소 컬럼을 SECURE_EXTENSION_DEGREE개의
// PackedBaseField 좌표 컬럼으로 분리합니다.
pub fn into_secure_column_by_coords(self) -> SecureColumnByCoords<SimdBackend>
if self.len() < N_LANES {
return self.to_cpu().into_iter().collect();
}

let length = self.length;
let packed_length = self.data.len();
let mut columns = array::from_fn(|_| Vec::with_capacity(packed_length));

for v in self.data {
let packed_coords = v.into_packed_m31s();
```

```
zip(&mut columns, packed_coords).for_each(|(col, packed_coord)| {
col.push(packed_coord)
});
}

SecureColumnByCoords {
columns: columns.map(|col| BaseColumn { data: col, length }),
}
}
}
```

캐시 친화적 접근 패턴

청크 기반 처리

메모리 접근 패턴을 캐시 라인에 맞게 최적화하기 위해 청크 단위로 데이터를 처리합니다:

```
impl BaseColumn {
/// BaseColumnMutSlice들의 벡터를 반환하며, 각각은 chunk_size개의
/// PackedBaseField들을 가변적으로 소유합니다.
pub fn chunks_mut(&mut self, chunk_size: usize) -> Vec<BaseColumnMutSlice<'_>
self.data
.chunks_mut(chunk_size)
.map(BaseColumnMutSlice)
.collect_vec()
```

```
}

}
```

병렬 청크 처리

보안 필드 컬럼에서는 병렬 청크 처리를 지원합니다:

```
impl SecureColumnByCoords<SimdBackend> {

#[cfg(feature = "parallel")]

pub fn par_chunks_mut(

&mut self,

chunk_size: usize,

) -> impl IndexedParallelIterator<Item = SecureColumnByCoordsMutSlice<'_>> {

let [a, b, c, d] = self.columns.each_mut().map(|c| c.chunks_mut(chunk_size));

(a, b, c, d)

.into_par_iter()

.map(|(a, b, c, d)| SecureColumnByCoordsMutSlice([a, b, c, d]))

}

}
```

고급 메모리 최적화

Very Packed 구조

더 높은 메모리 밀도를 위해 Very Packed 구조를 사용합니다:

```rust
pub const LOG_N_VERY_PACKED_ELEMS: u32 = 1;
pub const N_VERY_PACKED_ELEMS: usize = 1 << LOG_N_VERY_PACKED_ELEMS;

#[derive(Clone, Debug, Copy)]
#[repr(transparent)]
pub struct Vectorized<A: Copy, const N: usize>(pub [A; N]);

pub type VeryPackedM31 = Vectorized<PackedM31, N_VERY_PACKED_ELEMS>;
pub type VeryPackedQM31 = Vectorized<PackedQM31, N_VERY_PACKED_ELEMS>;
```

제로 카피 변환

메모리 복사를 피하기 위해 제로 카피 변환을 구현합니다:

```rust
impl BaseColumn {
pub fn into_cpu_vec(mut self) -> Vec<BaseField> {
let capacity = self.data.capacity() * N_LANES;
let length = self.length;
let ptr = self.data.as_mut_ptr() as *mut BaseField;
let res = unsafe { Vec::from_raw_parts(ptr, length, capacity) };
mem::forget(self);
res
}
```

```
}
```

타입 안전한 캐스팅

bytemuck 크레이트를 사용하여 타입 안전한 메모리 캐스팅을 수행합니다:

```
use bytemuck::{cast_slice, cast_slice_mut, cast_vec};


impl From<BaseColumn> for VeryPackedBaseColumn {
fn  from(value: BaseColumn) -> Self {
Self {
data: cast_vec(value.data),
length: value.length,
}
}
}
```

프리픽스 합 병렬 최적화

Blelloch 알고리즘 구현

프리픽스 합 계산은 Blelloch의 병렬 스캔 알고리즘을 사용하여 최적화됩니다:

```
pub  fn  inclusive_prefix_sum(
bit_rev_circle_domain_evals: Col<SimdBackend, BaseField>,
) -> Col<SimdBackend, BaseField> {
```

```
if bit_rev_circle_domain_evals.len() < N_LANES * 4 {
return inclusive_prefix_sum_slow(bit_rev_circle_domain_evals);
}

let mut res = bit_rev_circle_domain_evals;
let packed_len = res.data.len();
let (l_half, r_half) = res.data.split_at_mut(packed_len / 2);

// Up Sweep 단계 - 병렬 처리
for ([l0, l1], [r0, r1]) in izip!(
l_half.array_chunks_mut(),
r_half.array_chunks_mut().rev()
) {
let (mut half_coset0_lo, half_coset1_hi_rev) = l0.deinterleave(*l1);
let half_coset1_hi = half_coset1_hi_rev.reverse();
let (mut half_coset0_hi, half_coset1_lo_rev) = r0.deinterleave(*r1);
let half_coset1_lo = half_coset1_lo_rev.reverse();
up_sweep_val(&mut half_coset0_lo, half_coset1_lo);
up_sweep_val(&mut half_coset0_hi, half_coset1_hi);
*l0 = half_coset0_lo;
*l1 = half_coset0_hi;
*r0 = half_coset1_lo;
*r1 = half_coset1_hi;
```

```
}

// Down Sweep 단계 - 병렬 처리
// ... 유사한 병렬 처리 로직
res
}
```

메모리 접근 패턴 최적화

순차 접근 패턴

캐시 효율성을 위해 순차적 메모리 접근 패턴을 유지합니다:

```
impl FromIterator<BaseField> for BaseColumn {
fn from_iter<I: IntoIterator<Item = BaseField>>(iter: I) -> Self {
let mut chunks = iter.into_iter().array_chunks();
let mut data = (&mut chunks).map(PackedBaseField::from_array).collect_vec();
let mut length = data.len() * N_LANES;

if let Some(remainder) = chunks.into_remainder() {
let rem = remainder.len();
if rem > 0 {
length += rem;
let mut last = [BaseField::zero(); N_LANES];
```

```
last[..rem].copy_from_slice(remainder.as_slice());
data.push(PackedBaseField::from_array(last));
}
}

Self { data, length }
}
}
```

정렬된 메모리 접근

SIMD 연산을 위해 메모리 정렬을 보장합니다:

```
impl PackedM31 {
/// # Safety
/// 포인터가 PackedM31과 동일한 정렬을 가져야 함
pub const unsafe fn load(mem_addr: *const u32) -> Self {
Self(ptr::read(mem_addr as *const u32x16))
}
/// # Safety
/// 포인터가 PackedM31과 동일한 정렬을 가져야 함
pub const unsafe fn store(self, dst: *mut u32) {
ptr::write(dst as *mut u32x16, self.0)
}
```

```
}
```

확장성 설계

적응적 청크 크기

하드웨어 특성에 따라 최적 청크 크기를 사용합니다:

```
// Intel 155u 머신에서 경험적으로 결정된 최적 크기
pub(super) const PACKED_M31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 9; // 512
pub(super) const PACKED_CM31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 10; // 1024
pub(super) const PACKED_QM31_BATCH_INVERSE_CHUNK_SIZE: usize = 1 << 11; // 2048
```

조건부 최적화

데이터 크기에 따라 최적화 전략을 선택합니다:

```
impl ColumnOps<BaseField> for SimdBackend {
type Column = BaseColumn;

fn  bit_reverse_column(column: &mut  Self::Column) {
// 작은 데이터에 대해서는 CPU 비트 역순으로 폴백
if  column.data.len().ilog2() < MIN_LOG_SIZE {
cpu_bit_reverse(column.as_mut_slice());
return;
```

```
}

bit_reverse_m31(&mut  column.data);
}
}
```

메모리 풀링

대용량 증명 생성을 위해 메모리 재사용을 최적화합니다:

```
impl Column<BaseField> for BaseColumn {
fn  zeros(length: usize) -> Self {
let  data = vec![PackedBaseField::zeroed(); length.div_ceil(N_LANES)];
Self { data, length }
}
}
```

성능 향상 효과

병렬 처리 이득

- **멀티코어 활용**: Rayon을 통한 효율적인 작업 분산
- **작업 도용**: 동적 로드 밸런싱으로 CPU 활용률 극대화
- **NUMA 인식**: 메모리 지역성을 고려한 작업 스케줄링

메모리 최적화 효과

- **캐시 효율성**: 순차 접근 패턴으로 캐시 미스 최소화
- **메모리 대역폭**: 벡터화된 접근으로 대역폭 활용 극대화
- **메모리 사용량**: 패킹된 구조로 메모리 오버헤드 감소

확장성 특성

- **선형 확장**: 코어 수에 비례한 성능 향상
- **메모리 확장**: 대용량 증명에 대한 효율적인 메모리 관리
- **적응적 최적화**: 하드웨어 특성에 따른 자동 최적화

결론

Stwo의 병렬 처리 및 메모리 최적화는 Circle STARK 증명 시스템의 확장성과 성능을 크게 향상시킵니다. Rayon을 활용한 효율적인 병렬 처리, 캐시 친화적인 메모리 접근 패턴, 그리고 하드웨어 특성을 고려한 적응적 최적화를 통해 대규모 증명 생성에서 실질적인 성능 향상을 제공합니다. 이러한 최적화 기법들은 현대 멀티코어 시스템의 성능을 최대한 활용하여 실용적인 STARK 증명 시스템을 구현합니다.